2.0 Intérprete, variables y tipos de datos

Programación I
Ingeniería Electrónica y Telecomunicaciones

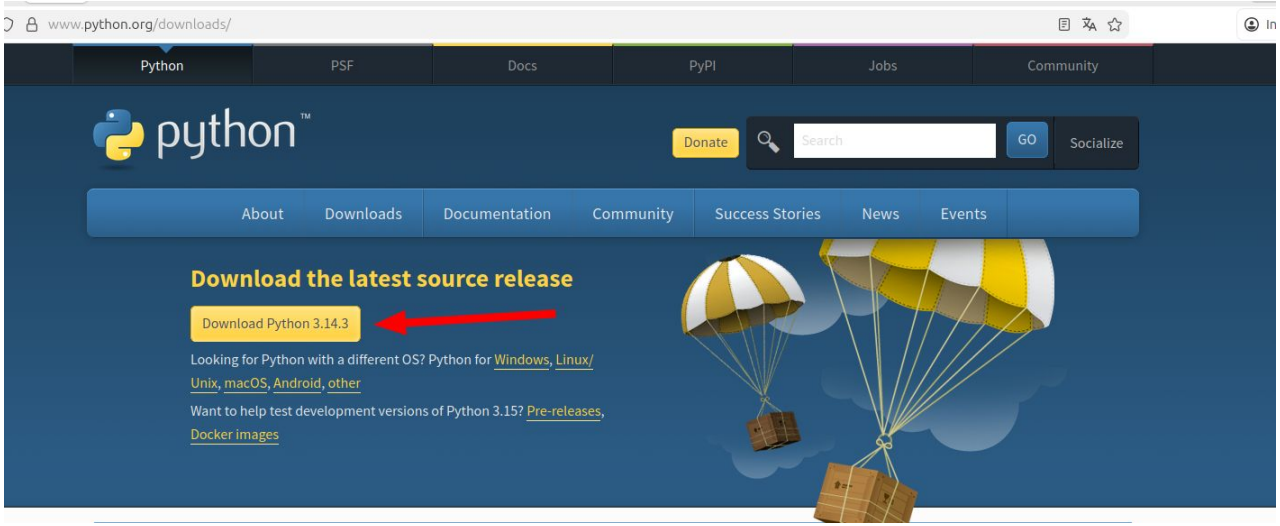
Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Rio Negro

Repaso express de la clase anterior

- ¿Qué es hardware?
- ¿Qué es un sistema operativo?
- ¿Qué es una aplicación?
- ¿Qué lenguaje vamos a usar?
  - ¿es compilado o interpretado?
- ¿Alguien se acuerda algo de linux y bash?

Instalación de python

- Instalar python: https://www.python.org/downloads/



- Instalar python en WSL (Para los que hicieron la tarea):
  - sudo apt update && sudo apt upgrade
  - sudo apt install python3 python3-pip python3-venv build-essential -y

Intérprete de Python

Python se distribuye junto con una consola que nos permite ejecutar instrucciones de forma manual.

Puede ser usado de calculadora, también es muy útil para hacer pequeñas pruebas y nos permite jugar didácticamente directamente con python sin necesita de un archivo.

Podemos iniciar la consola de python de las siguientes maneras:

- Linux o WSL podemos ejecutar en una terminal el comando: **python o python3**
- Windows podemos simplemente buscar python en el inicio

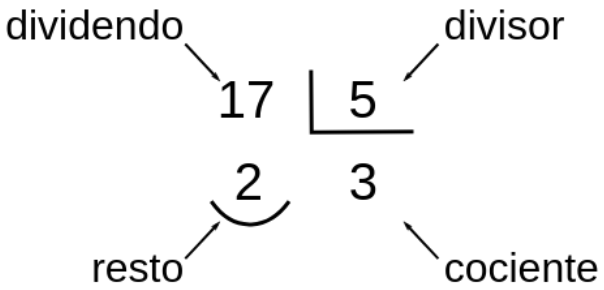
Intérprete de Python: Cosas utiles

- El shell interactivo se ve así: “>>>”
- print() para imprimir
- help() para obtener ayuda
- quit() para salir
- Puede que nos encontremos con “...” durante la ejecución
    - Podemos escapar de esto con CTRL + C

Intérprete de Python: Usando como calculadora

Operador	Nombre
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo
**	Exponente
//	Cociente

dividendo = divisor * cociente + resto



- El orden de las operaciones aritméticas en Python sigue la regla **PEMDAS** (Paréntesis, Exponentes, Multiplicación/División, Suma/Resta), evaluando de izquierda a derecha para operadores con igual jerarquía.
  - (): Paréntesis (máxima prioridad).
  - **: Exponenciación.
  - *, /, //, %: Multiplicación, división, división de piso, modulo
  - +, -: Suma y resta

Variables

Es una pieza clave en el desarrollo de software que permite que el software recuerde el resultado de operaciones.
Se almacenan en la memoria RAM de la computadora.

Las variables nos permiten almacenar valores con un nombre asociado. (**Está operación la llamamos asignación**)

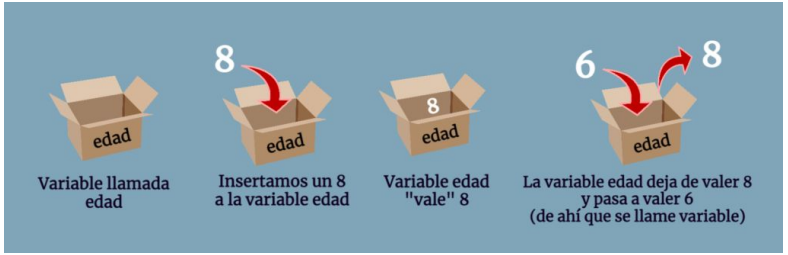
```
variable_x = 10
```

Las variables se pueden modificar durante la ejecución de un programa.

```
variable_x = 15
```

Las variables pueden ser eliminadas

```
del variable_x
```



Variables - Regla de nombres

- Las variables son case sensitive, es decir, si se definen en minúscula, se deben acceder de la misma manera.
- Se pueden usar letras de la A a la Z, números y guiones.
- No pueden comenzar con número.
- No pueden contener símbolos extraños, por eso evitamos los acentos, las Ñ y otros símbolos extraños.
  - Consejo: Para escribir año utilicen el nombre “anio”, es más bonito :smile:
- Existen restricciones de nombres en todos los lenguajes, normalmente llamados “Palabras Reservadas”.
  - En python podemos revisarlas con el s
    >>> help("keywords")

```
>>> help("keywords")

Here is a list of the Python keywords.  Enter any keyword to get more help
.

False               class               from                or
None                continue            global              pass
True                def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
break               for                 not
```

Variables - Buenas Prácticas

Las variables nos pueden ayudar a entender como funciona nuestro programa.

El nombre de una variable lo elige el programador, pero es recomendable usar nombres descriptivos que ayuden a entender qué representa ese dato.

Buenos nombres es igual a menos dolores de cabeza cuando tengamos que ver nuestro código y entender que hace.

Cuando trabajamos en equipo, nuestros compañeros necesitan entender nuestro programa.

Cuando volvamos a nuestro programa dentro de 6 meses, tenemos que seguir entiendo que hace nuestro programa.

```
a = 25
b = 10
c = a * b
```

```
precio = 25
cantidad = 10
total = precio * cantidad
```

Tipos de variables en python

Los tipos de datos son una parte fundamental de cualquier lenguaje de programación, ya que definen qué tipo de valores pueden almacenar las variables y cómo se pueden manipular esos valores.

En esta introducción vamos a conocer los tipos:

- Booleanos (bool): Nos permiten almacenar un estado Verdadero o Falso
- Enteros (int): Nos permiten almacenar números enteros.
- Flotantes (float): Nos permiten almacenar números decimales.
- Cadenas de texto (string): Nos permiten almacenar texto.

Cada uno tiene sus particulares y operaciones.

Entero - Int

La variable de tipo entero nos permite almacenar valores numéricos con signo, es decir, valor positivos y negativos.

Lo podemos usar para contar cosas por ejemplo cantidad de frutas o años.

EJEMPLOS

mi_edad = 30

altura_en_centimetros = 183

cantidad_de_alumnos = 39

saldo_cuenta_banco = -500000

OPERACIONES

Nombre	Operador	Ejemplo
Resta	-	**mi_edad** = 2026 - 1996
Suma	+	**anio_actual** = 1996 + 30
Multiplicar	*	**semanas_anio** = 12 * 4
División entera	//	**caramelos_por_nino** = 10 // 3
Modulo	%	**caramelos_sobrantes** = 10 % 3
Potencia	**	**x = 1 ** 2**

type(): El ayudante de tipos

La función type nos ayuda a entender con qué de variables estamos trabajando.

Prueben en sus terminales

>> type(1)

Pueden usarlo en cada proximo paso para entender mejor qué está pasando con las variables.

Entero - Int

Diferentes representaciones.

OJO: HEX y BIN devuelven texto.

EJEMPLOS

mi_edad_dec = 30
mi_edad_hexa = 0x1E
mi_edad_binaria = 0b11110

cantidad_de_alumnos_dec = 39
cantidad_de_alumnos_hexa = 0x29
cantidad_de_alumnos_binario = 0b100111

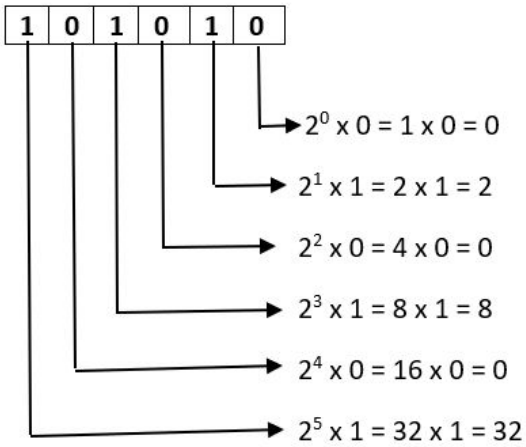
OPERACIONES

Nombre	Func	Ejemplo
Conv Int To Hex	hex(n)	**mi_edad_hexa** = hex(30)
Conv Int to Bin	bin(n)	**mi_edad_binaria** = bin(30)
Conv Str to Int	int(n_str, base)	**mi_edad_dec** = int("0x1E", 16)

Entero – Int
Pasaje de bases

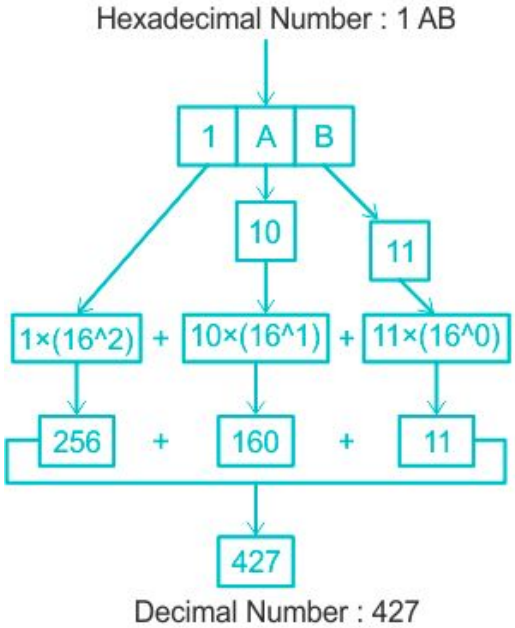
Decimal	Binary	Hexadecimal
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Bin to Int



Resultant decimal number= 0+2+0+8+0+32 = **42**

Hex to int



Flotante - Float

A diferencia de int, que sirve para contar, los float sirven para medir.

El float nos permite representar números negativos y positivos con decimales.

EJEMPLOS

temperatura_dia = 18.5
temperatura_noche = -5.4
altura = 1.72
precio = 10.99
peso_auto = 3000.50
peso_persona = 92.43

OPERACIONES:

Nombre	Operador	Ejemplo
Resta	-	**amplitud_termica** = 18.5 - -5.4
Suma	+	**peso_total** = 3000.50 + 92.43
Multiplicar	*	**area** = 3.5 * 3.5
División	/	**velocidad** = 10 / 3
Modulo	%	**x** = 10 % 3.5

Flotante - Float

Representación de los floats y curiosidades

Representación

```
print(18.523451) # imprimiento flotante
print(f"{18.523451:.2f}") # Forzamos a 2 la cantidad decimales
print(f"{18.523451:.2f}") # Forzamos a 2 la cantidad decimales
print(f"{18.523451:08.3f}") # Forzamos que siempre se muestres 8 digitos
```

Curiosidad: Python permite operar int y float en las mismas operaciones, cuando se realizan este tipo de operaciones python transforma la salida siempre a tipo float.

Prueben lo siguiente:

```
>>> 1 + 2
>>> 1 + 2.0
```

Puede ejecutarlo junto a la sentencia type para verificarlo.

Cadena de texto - string

Las variables tipo string nos permiten almacenar texto.

Siempre que asignemos variables debemos usar comillas dobles o simples (“ o ‘)

El libre y podemos poner numeros, simbolos y hasta emoticones.

EJEMPLOS:

```
nombre = "Ana"
mensaje = "Hola mundo"
ciudad = "Bariloche"

parrafo = """
Yo vivo en bariloche.
Estudio ingenieria
""" # Se permiten textos multilinea
```

OPERACIONES:

Nombre	**Operador**	**Ejemplo**
Suma	+	**texto** = “Hola “ + “Mundo “ + “Cruel”
Multiplicar	*	**mensaje_ayuda =** “!Ayuda! ” * 10
Acceso por índice	[n]	letra = “Hola Mundo”**[0]**
Último carácter	[-1]	ultima_letra = “Hola Mundo”**[-1]**
Cantidad letras	len	cantidad_letras = len(“hola mundo”)

Cadena de texto - string

Representación de textos y convivencia con los números

EJEMPLOS:

print(“hola” + “ mundo”) # Concatenación normal

print(“Tu nota es un ” + str(10)) # Concatenación con numeros

print(“Tu nota es un ” + str(4.2)) # Concatenación con numeros

var = “Juan”
nota = 4.2
print(f“Hola {var}, tu nota es {nota}”) # Formateo con variables

Más operaciones

Nombre	Operador	Ejemplo
Centrar	center	“Hola mundo cruel”.center(“-”, 20)
Contador	count	“Banana”.count("a")
Remplazar	replace	“Hola mundo”.replace(“Hola”, “Holis”)

Bool

La variable booleana permite almacenar solamente dos estados.

Verdadero y Falso.

Los operadores booleanos los dejamos para más adelante.

```
variable_falsa = False

variable_verdadera = True

llueve_afuera = False

hay_sol = True

variable_verdadera_1 = bool(1)

variable_falsa_2 = bool(0)
```

Programas python persistentes

Anteriormente vimos como utilizar el interprete de linea de comandos de python.

Ésto sirve para probar cosas pero no nos permite almacenar un programa que podamos ejecutar posteriormente.

Para almacenar un programa python tenemos que abrir un editor de texto, puede ser Notepad, VsCode o Nano.

1. Escriban print(“Hola Mundo”)
2. Vayan a “Guardar Como”.
   a. GUARDAR COMO “holamundo.py”
   b. **IMPORTANTE:** Todos los programas python deben terminar en “.py”
3. Abrir una terminal, sea bash o powershell y ejecutar con el comando “python holamundo.py”

Vamos con un minitp

TAREA

1. Para quienes no lo hicieron, completar la tarea de la clase 1:
   https://github.com/Ymil/unrn-programacion-1/tree/main/clases/clase-01
2. Hacer tarea de la clase 2
   https://github.com/Ymil/unrn-programacion-1/tree/main/clases/clase-02
   a. Instalar python en sus casas los que no pudieron
   b. Realizar los ejercicios de la guia 1 (mini tp1)

Todo el material en https://github.com/Ymil/unrn-programacion-1/

FIN

Volvemos el miércoles a las 17